

Iterables

Los **iterables** son elementos que **pueden recorrerse de manera iterativa**. Las **matrices son iterables**, así como los **Strings** y muchos **elementos similares a matrices** (como **arguments**). Otros objetos que veremos posteriormente son también iterables (**Map**, **Sets**, **NodeList**).

Sobre los elementos iterables **se puede ejecutar** el bucle **for ... of**:

```
for (let v of [1,3,5,7]) {
```

```
  console.log(v);
```

```
}
```

```
>1
```

```
>3
```

```
>5
```

```
> 7
```

```
for (let c of "Hola") {
```

```
  console.log(c);
```

```
}
```

```
>H
```

```
>o
```

```
>l
```

```
>a
```

Map

Un **Map** es una estructura que nos permite almacenar pares de clave / valor. Es similar a un objeto pero con la diferencia de que las claves pueden ser de cualquier tipo (y no necesariamente **Number**, **Strings** y/o **Symbol** como en el caso de los objetos). Otra diferencia es que las claves en el **Map** están ordenadas. (En la práctica se utilizan más los objetos que los mapas)

Constructor

- **Map([iterable])** : Crea un objeto **Map** (opcionalmente a partir de un iterable que retorne objetos de tipo clave/valor)

Propiedades

- **length** : Retorna el número de elementos actuales del mapa

Métodos

- **clear()** : Elimina todos los elementos del mapa
- **delete(clave)** : Elimina el elemento con la **clave** indicada

- `entries()` : Devuelve un **iterador** con todas las entradas (clave/valor) del mapa
- `forEach(func[,objThis])` : Ejecuta la **función** indicada para **cada valor** del mapa. Si se proporciona además un objeto, éste se asigna a **this** dentro de la función. La función recibe como parámetro la entrada correspondiente del **Map**
- `get(clave)` : Retorna el **valor asociado** a la **clave** indicada
- `has(clave)` : Retorna **true** si hay un elemento en el mapa con dicha **clave**
- `keys()` : Retorna un **iterador** con las **claves** del mapa
- `set(clave,valor)` : **Añade** un nuevo elemento al mapa **si ningún elemento tiene esa clave o actualiza** el **valor** del elemento con la **clave** dada
- `values()` : Retorna un **iterador** con los **valores** del mapa

```
let m = new Map(); // Otra forma seria: (se inicializa con un array de arrays)
                  // donde cada subarray debe tener dos posiciones, 0 clave, 1 valor
m.set("es", "Español"); // let n=new Map([["es", "Español"],["en", "Ingles"], ["de", "Aleman"]]);
m.set("en", "Ingles");   // n
m.set("de", "Aleman");   // > Map(3) {'es' => 'Español', 'en' => 'Ingles', 'de' => 'Aleman'}
```

```
m
>Map(3) {"es" => "Español", "en" => "Inglés", "de" => "Alemán"}
```

```
for(let e of m){
  console.log(`${e[0]} : ${e[1]}`);
}
```

```
>es : Español
```

```
>en : Inglés
```

```
>de : Alemán
```

```
for(let e of m.entries()){
  console.log(`${e[0]} : ${e[1]}`);
}
```

```
>es : Español
```

```
>en : Inglés
```

```
>de : Alemán
```

```
for(let e of m.keys()){
  console.log(`${e} : ${m.get(e)}`);
}
```

```
>es : Español
```

```
>en : Inglés
```

```
>de : Alemán
```

```
m.forEach(function(e){
```

```
    console.log(`${e.toUpperCase()}`);
```

```
})
```

```
>ESPAÑOL
```

```
>INGLÉS
```

```
>ALEMÁN
```

```
m.set("fr", "Francés");
```

```
m.set("de", "Germánico");
```

```
m.delete("de");
```

```
console.log(Array.from(m).join("\n")); (Esta pasando de mapa a array y luego a cadena  
con join, separando con \n)
```

```
>es, Español
```

```
en, Inglés
```

```
fr, Francés
```

Set

Un Set es un conjunto único de valores (es decir, no puede repetirse ninguno).

Constructor

- `Set([iterable])` : Crea un objeto Set (opcionalmente a partir de un iterable).

Es práctico por ejemplo para quitar los repetidos de un array:

```
let numeros = [1,1,1,2,3,4]
```

```
> undefined
```

```
let conjuntoSinRepetidos = new Set(numeros)
```

```
> undefined
```

```
let arraySinRepetidos = Array.from(conjuntoSinRepetidos)
```

```
> undefined
```

```
numeros
```

```
> (6) [1, 1, 1, 2, 3, 4]
```

```
conjuntoSinRepetidos
```

```
> Set(4) {1, 2, 3, 4}
```

```
arraySinRepetidos
```

```
> (4) [1, 2, 3, 4]
```

Propiedades

- `size` : Retorna el **número de elementos** actuales del set

Métodos

- `add(elem)` : **Añade** un nuevo elemento al conjunto **si no existe**
- `clear()` : **Elimina todos** los elementos del conjunto
- `delete(elem)` : **Elimina el elemento** indicado del conjunto **si existe**
- `entries()` : Retorna un **iterador de pares valor/valor con los elementos** del conjunto
- `forEach(func[,objThis])` : Ejecuta la función indicada **para cada valor** del conjunto. Si se proporciona además un objeto, éste se asigna a **this** dentro de la función
- `has(elem)` : Retorna **true** si hay un elemento en el conjunto con dicho valor

- `values()` : Retorna un **iterador con los valores** del conjunto

```
let usuarios = new Set([ "juan", "luis", "julio" ]);
for(let u of usuarios){
    console.log(u);
}
>juan
>luis
>julio
//add añade un nuevo elemento, si ya no existe
usuarios.add("ana");
usuarios.add("luis");
//usuarios.entries(): Retorna un iterador de pares valor/valor con los elementos del
conjunto
for(let u of usuarios.entries()){
    console.log(u);
}
>(2) ["juan", "juan"]
>(2) ["luis", "luis"]
>(2) ["julio", "julio"]
>(2) ["ana", "ana"]
//has(elem): Retorna true si hay un elemento en el conjunto con dicho
valor
if (usuarios.has("luis")){
    usuarios.delete("luis");
}
for(let u of usuarios.values()){
    console.log(u);
}
> juan
> julio
> ana
```

```

usuarios.forEach((item) => {
    if (item.startsWith("j")){
        usuarios.delete(item);
    }
});

//values() : Retorna un iterador con los valores del conjunto
for(let u of usuarios.values()){
    console.log(u);
}

> ana

```

Iterables y Object

Adicionalmente disponemos en la clase **Object** de métodos que nos permiten conversiones entre objetos y mapas:

- **Object.entries(objeto)** : Retorna un iterador con las propiedades del objeto como clave/valor del mismo
- **Object.fromEntries(iterador)** : Retorna un objeto con las entradas del iterador como propiedades del mismo
- **Object.keys(objeto)** : Retorna un iterador con los nombres de las propiedades del objeto
- **Object.values(objeto)** : Retorna un iterador con los valores de las propiedades del objeto como clave/valor del mismo

```

let usuario = {
    nombre:"juan",
    edad:18,
}

// Retorna un objeto con las entradas del iterador como propiedades del mismo
console.log(Object.entries(usuario));

>(2) [Array(2), Array(2)]

//Retorna un iterador con los nombres de las propiedades del objeto

```

```

console.log(Object.keys(usuario));
>(2) ["nombre", "edad"]
//Retorna un iterador con los valores de las propiedades del objeto
como clave/valor del mismo
console.log(Object.values(usuario));
>(2) ["juan", 18]
console.log(new Map(Object.entries(usuario)));
>Map(2) {"nombre" => "juan", "edad" => 18}
console.log(new Set(Object.keys(usuario)));
>Set(2) {"nombre", "edad"}
let set = new Set(["Uno", "Dos", "Tres"]);
//Object.fromEntries(iterador) : Retorna un objeto con las entradas
del iterador como propiedades del mismo
console.log(Object.fromEntries(set.entries()));
>{Uno: "Uno", Dos: "Dos", Tres: "Tres"}
let map = new Map(
);
console.log(Object.fromEntries(map.entries()));
>{1: "Uno", 2: "Dos", 3: "Tres"}

```

Desestructuración de Objetos, Matrices e Iterables

La desestructuración de objetos permite extraer las propiedades de los objetos a variables:

Por ejemplo (partiendo del objeto `usuario` creado):

```

let usuario = {
  nombre:"juan",
  edad:18,
  pais: "España",
  altura: 175
}

```

OJO! Las variables de deben llamar igual que las propiedades

```

let { nombre, edad } = usuario;
> undefined
nombre
> "juan"

```

```
edad  
> 18
```

Algo similar se puede hacer con matrices e iterables:

Por ejemplo:

```
let mat = [1,2,3,4,5];  
let [uno,dos,...resto] = mat;  
console.log(`${uno} : ${dos} : ${resto}`);  
>1 : 2 : 3,4,5  
let set = new Set([1,2,3,4,5,6]);  
let [a,b,...r] = set;  
console.log(`${a} : ${b} : ${r}` )  
> 1 : 2 : 3,4,5,6
```